

Nyx

A Data-Driven Astronomical Observation Platform Using NASA Space Data
and Local Observations in Mauritius

Zakariyya Kurmally

Student ID: 2315839

BSc (Hons) Data Science

University of Mauritius

Supervisor: Mr Somveer Kisna

2026

Contents

Glossary	4
1. Introduction	6
1.1. Background	6
1.2. Problem Statement	7
1.3. Aim and Objectives	8
1.4. Scope and Limitations	9
1.5. Report Structure	9
2. Literature Review	9
2.1. Astronomical Data Sources	10
2.1.1. The TESS Mission	10
2.1.2. The FITS Data Format	10
2.1.3. The MAST Archive	11
2.2. Existing Platforms and Tools	11
2.2.1. Lightkurve	11
2.2.2. Exoplanet Transit Database	12
2.2.3. AAVSO International Database	12
2.2.4. ExoFOP	12
2.3. Data Pipeline Architecture	13
2.3.1. ETL and ELT Patterns	13
2.3.2. Batch vs. Real-Time Processing	13
2.4. Software Architecture	14
2.4.1. Clean Architecture	14
2.4.2. Hexagonal Architecture	14
2.5. Backend Technologies	15
2.5.1. C++ for Web Services	15
2.5.2. Drogon Web Framework	15
2.6. Database Design for Time-Series Data	15
2.6.1. PostgreSQL for Scientific Data	15
2.6.2. Alternatives Considered	16
2.7. Authentication and Security	16
2.7.1. JWT-Based Authentication	16
2.7.2. OAuth 2.0	17
2.7.3. CSRF Protection	17
2.7.4. Password Hashing	17
2.8. API Design	18
2.8.1. REST Principles	18
2.8.2. Cursor-Based Pagination	18
2.9. Frontend Technologies	18
2.9.1. Next.js	18
2.9.2. Data Visualisation	18
2.10. Summary	19
3. Requirements Analysis	19
4. System Design	19

5. Implementation	19
6. Testing and Validation	19
7. Conclusion	19
Bibliography	20

Glossary

- API** Application Programming Interface. A set of defined rules enabling software components to communicate.
- BJD** Barycentric Julian Date. A time standard that corrects for the observer's position relative to the Solar System barycentre, removing orbital motion effects from timing measurements.
- BTJD** Barycentric TESS Julian Date. The time system used by TESS, corrected for light travel time to the Solar System barycentre. Defined as $\text{BJD} - 2457000.0$.
- CAOM** Common Archive Observation Model. A standardised data model used by MAST to describe astronomical observations across multiple missions.
- CSRF** Cross-Site Request Forgery. An attack where a malicious site submits requests on behalf of an authenticated user. Mitigated using the double-submit cookie pattern.
- ETL** Extract, Transform, Load. A data pipeline pattern for ingesting data from external sources, transforming it into a target format, and loading it into a database.
- FFI** Full-Frame Image. A TESS data product capturing the entire field of view of a camera at 30-minute (primary mission) or 10-minute (extended mission) cadence.
- FITS** Flexible Image Transport System. The standard file format in astronomy for storing images, spectra, and tabular data. Defined by the IAU FITS Working Group.
- Flux** The amount of light received from an astronomical object per unit area per unit time, typically measured in electrons per second (e^-/s) for space-based photometry.
- HDU** Header Data Unit. The fundamental building block of a FITS file, consisting of an ASCII header followed by an optional data array.
- JWT** JSON Web Token. A compact, URL-safe token format used for stateless authentication. Contains encoded claims signed with a secret key.
- Light Curve** A time-series of flux measurements for an astronomical object, used to detect transits, eclipses, and variability.
- MAST** Mikulski Archive for Space Telescopes. NASA's primary archive for ultraviolet, optical, and near-infrared astronomical data, hosted by the Space Telescope Science Institute.
- OAuth2** Open Authorization 2.0. An industry-standard protocol for delegated authentication, allowing users to sign in via third-party providers such as Google.
- PDCSAP Flux** Pre-search Data Conditioning Simple Aperture Photometry flux. TESS flux values corrected for systematic errors such as spacecraft pointing jitter and thermal variations.

Photometry The measurement of the intensity of light from astronomical objects. Differential photometry measures brightness changes relative to comparison stars in the same field.

RAII Resource Acquisition Is Initialisation. A C++ idiom where resource lifetime is tied to object lifetime, ensuring deterministic cleanup via destructors.

REST Representational State Transfer. An architectural style for web APIs using standard HTTP methods (GET, POST, PUT, DELETE) with stateless request-response semantics.

SAP Flux Simple Aperture Photometry flux. Raw flux values extracted from TESS pixel data by summing counts within a defined aperture, before systematic correction.

SPOC Science Processing Operations Center. The NASA Ames facility responsible for processing raw TESS data into calibrated light curves and target pixel files.

SSE Server-Sent Events. A unidirectional HTTP-based protocol for pushing real-time updates from server to client over a persistent connection.

SSR Server-Side Rendering. A web rendering strategy where HTML is generated on the server rather than in the browser, improving initial load performance.

TESS Transiting Exoplanet Survey Satellite. A NASA space telescope launched in 2018 that surveys nearly the entire sky for exoplanet transits using transit photometry.

TOI TESS Object of Interest. A star identified by TESS as hosting a candidate transiting planet, pending confirmation by follow-up observations.

TPF Target Pixel File. A TESS data product containing the raw pixel data for a small region around a target star, used for custom aperture photometry.

Transit The passage of a planet in front of its host star as seen from the observer, causing a measurable dip in the star's brightness.

1. Introduction

1.1. Background

Modern astronomy is mostly a data science field. The amount of data from space missions has been growing at an exponential rate in the last two decades. Missions like the Kepler mission and its successor the Transiting Exoplanet Survey Satellite (TESS) have contributed to this growth. The Kepler mission, which launched in 2009, observed 150,000 stars for four years and led to the discovery of the first large number of exoplanets (Borucki *et al.*, 2010). The TESS mission, which launched in 2018, has observed 26 different areas of the sky for approximately 27 days each (Ricker *et al.*, 2015). TESS collects 27 GB of data per orbit, enabling high-cadence brightness measurements of hundreds of thousands of stars per orbit. All of the data collected by TESS is stored in NASA’s Mikulski Archive for Space Telescopes (MAST). MAST stores data from numerous astronomical missions, with over 500 TB of stored data as of 2024 (Space Telescope Science Institute, 2024).

The primary technique underlying both missions is transit photometry: detecting the periodic dimming of a star’s brightness as a planet passes in front of it. The resulting data product is a light curve — a time-series of flux (brightness) measurements. The depth of a transit dip reveals the planet’s radius relative to the star, while the period between dips gives the orbital period (Winn, 2010). Light curves are also used to study stellar variability, eclipsing binary systems, and other astrophysical phenomena. The ability to analyse these time-series datasets is central to modern observational astronomy.

Simultaneously, ground-based amateur astronomy has become increasingly capable. Affordable reflecting telescopes, motorised equatorial mounts, and open-source data reduction software have enabled amateur astronomers to produce scientifically useful photometric measurements (Fitzgerald *et al.*, 2014). Even entry-level instruments such as a 130mm Newtonian reflector on an equatorial mount, paired with a smartphone camera or a dedicated CMOS sensor, can capture images suitable for differential photometry of bright variable stars and exoplanet transits (Gary, 2007). Organisations such as the American Association of Variable Star Observers (AAVSO) and the British Astronomical Association (BAA) routinely incorporate amateur observations into professional research programmes, particularly for variable star monitoring and exoplanet transit confirmation (Conti and Conti, 2018). Ground-based observations complement space-based data in several ways: they can provide longer temporal baselines, fill gaps between space mission observation windows, and offer independent confirmation of detected signals.

However, a disconnect exists between these two data sources. Space-based and ground-based observations differ in cadence, precision, noise characteristics, and data formats. TESS observations are stored in FITS (Flexible Image Transport System) files — a binary format standard in astronomy — containing calibrated flux measurements at 2-minute or 30-minute intervals (Tenenbaum and Jenkins, 2018). These files use the Barycentric TESS Julian Date (BTJD) time system, which corrects for the satellite’s orbital motion relative to the Solar System barycentre (Eastman, Siverd and Gaudi, 2010). Ground-based amateur observations are typically captured as JPEG or FITS images requiring photometric

extraction before analysis, and use UTC or Julian Date time systems. Integrating these heterogeneous sources into a unified pipeline requires parsing different binary formats, reconciling time systems, handling different noise profiles (systematic spacecraft trends vs. atmospheric scintillation), and normalising flux scales — a non-trivial data engineering challenge.

Mauritius, located at 20.3°S latitude and 57.5°E longitude in the Indian Ocean, offers several geographic advantages for ground-based astronomy. First, its southern hemisphere position provides visibility of targets that are less accessible from major Northern Hemisphere observatories, including the Magellanic Clouds, the Galactic Centre, and southern-sky exoplanet hosts. This is particularly relevant to TESS, which devoted its first year of operations (July 2018 – July 2019) to the southern ecliptic hemisphere before surveying the north (Ricker *et al.*, 2015), meaning the densest TESS coverage aligns with the sky visible from Mauritius. Second, Mauritius occupies a longitude gap between major professional observatory sites. The bulk of southern hemisphere observing capacity is concentrated in Chile (70°W), South Africa (18°E), and eastern Australia (150°E) (Hessman, 2004). An observer at 57.5°E can capture data during nighttime hours when none of these three sites are observing, providing temporal coverage that would otherwise be missing from ground-based follow-up campaigns. Despite these advantages, there is limited astronomical observation infrastructure on the island, and no existing platform that enables local observers to systematically compare their observations against space-based data.

1.2. Problem Statement

NASA’s open data archives provide high-quality photometric time-series data, but they are designed primarily for professional researchers. The MAST API uses a columnar JSON response format that requires domain-specific knowledge to query and interpret. A single MAST query involves constructing a JSON request specifying service name, parameters, and data filters, then parsing a response where column names, types, and values are returned as separate arrays rather than as conventional row-oriented records (Space Telescope Science Institute, 2024). For an observer who wants to answer a simple question — “how does my ground-based measurement of star X compare to what TESS observed?” — the current workflow involves manually querying MAST, downloading FITS files, parsing binary table extensions using libraries such as CFITSIO (Pence *et al.*, 2010), converting between BTJD and Julian Date time systems, and plotting the results using scripts or specialised tools such as Lightkurve (Lightkurve Collaboration *et al.*, 2018). This multi-step process requires proficiency in Python, familiarity with astronomical data formats, and knowledge of the MAST API — a significant barrier for amateur observers.

Several existing tools partially address this problem. Lightkurve provides a Python API for downloading and analysing Kepler and TESS light curves, but it is a programmatic library, not a web platform, and offers no mechanism for integrating user observations (Lightkurve Collaboration *et al.*, 2018). The Exoplanet Transit Database (ETD) allows amateurs to submit transit observations, but does not provide direct access to TESS data or automated comparison (Poddany, Brat and Pejcha, 2010). The AAVSO International Database accepts

photometric submissions but is focused on variable star monitoring rather than exoplanet transit analysis, and does not offer integrated visualisation of space-based reference data (Kafka, 2021).

No existing platform provides an integrated pipeline that:

- Resolves astronomical targets by name against authoritative catalogues.
- Retrieves and caches TESS observation metadata and light curve data.
- Parses FITS binary tables into structured, queryable time-series stored in a relational database.
- Exposes this data through a REST API suitable for frontend visualisation.
- Allows users to upload their own ground-based observations and overlay them against space-based data for comparison.

This gap is not primarily an astrophysics problem — it is a data engineering problem. The scientific data already exists. The challenge lies in building a performant, well-architected pipeline that ingests, transforms, stores, and serves heterogeneous astronomical data from multiple sources.

1.3. Aim and Objectives

The aim of this project is to design and implement a data-driven astronomical observation platform that integrates NASA space mission data with ground-based observations captured from Mauritius.

The objectives are:

1. Design and implement a backend using modern C++ that exposes a RESTful API for synchronous operations, background job processing for long-running data ingestion tasks, and Server-Sent Events (SSE) for real-time progress updates to the frontend. The architecture should follow clean architecture principles with clear separation between domain logic, data access, and presentation layers.
2. Build a data ingestion pipeline that queries NASA’s MAST API, retrieves TESS observation metadata, discovers data products, and downloads and parses FITS light curve files into structured time-series data.
3. Implement a PostgreSQL-backed persistence layer that stores astronomical targets, TESS observations, and light curve data points in a schema optimised for time-series queries.
4. Develop an authentication and authorisation system supporting local registration with email verification and third-party OAuth2, with security measures including JWT token rotation, CSRF protection, and rate limiting.
5. Create an observation management module that allows authenticated users to record ground-based observation sessions with associated equipment metadata and captured images.
6. Build a frontend application that provides interactive light curve visualisation, a target catalogue browser, and data overlay capabilities for comparing space-based and ground-based observations.

7. Validate the system by ingesting real TESS data for known exoplanet host stars and comparing the resulting light curves against published values.

1.4. Scope and Limitations

This project covers the full-stack development of the Nyx platform, from database schema design through API implementation to frontend visualisation. The primary data source is NASA’s TESS mission, accessed via the MAST API. The system is designed to be extensible to other NASA data sources such as the Exoplanet Archive, but integration with additional archives is not a primary deliverable.

The following are explicitly out of scope:

- **Real-time telescope control.** The platform manages observation data after capture, not during acquisition.
- **Automated plate-solving or astrometry.** Users specify their target by name; the system does not attempt to identify targets from image content.
- **Advanced photometric reduction.** The system accepts pre-reduced photometric measurements from ground-based observations. Full aperture photometry from raw images is not implemented.
- **Multi-site deployment.** The system is designed for single-instance deployment. Distributed architecture and horizontal scaling are not addressed.

The default observation site is Mauritius (longitude 57.5°E, latitude 20.3°S), which is used for visibility calculations and observation planning features.

1.5. Report Structure

The remainder of this report is organised as follows. Section 2 reviews existing astronomical data platforms, data pipeline architectures, and relevant technologies, establishing the academic and technical context for the project. Section 3 presents the requirements analysis, including functional and non-functional requirements derived from the problem statement and stakeholder needs. Section 4 describes the system architecture, database schema design, API design, and key design decisions with their rationale. Section 5 details the technical implementation of each major subsystem, including the data ingestion pipeline, authentication system, and frontend application. Section 6 covers the testing strategy, including unit tests, integration tests, and validation against known astronomical data. Section 7 evaluates the project against its objectives, discusses lessons learned, and identifies future work.

2. Literature Review

This chapter reviews the academic and technical context underpinning the Nyx platform. It covers the astronomical data sources and formats that the system must ingest, existing platforms that partially address the problem space, data pipeline architectures relevant to the ingestion workflow, software architecture patterns applied in the backend design,

database strategies for time-series astronomical data, authentication and security mechanisms, API design principles, and frontend technologies for data visualisation.

2.1. Astronomical Data Sources

2.1.1. The TESS Mission

The Transiting Exoplanet Survey Satellite (TESS) is a NASA Astrophysics Explorer mission designed to detect exoplanets via the transit method across nearly the entire sky (Ricker *et al.*, 2015). Unlike its predecessor Kepler, which stared at a single 115-square-degree field for four years (Borucki *et al.*, 2010), TESS divides the sky into 26 overlapping sectors of $24^\circ \times 96^\circ$ each, observing each sector for approximately 27.4 days. The spacecraft carries four wide-field CCD cameras with a combined field of view of $24^\circ \times 96^\circ$, covering 2300 square degrees per sector (Ricker *et al.*, 2015).

TESS produces two categories of data products. Full-Frame Images (FFIs) capture the entire field of view at 30-minute cadence (reduced to 10 minutes in the extended mission), providing photometry for millions of stars per sector. Target Pixel Files (TPFs) and Light Curve Files are produced for pre-selected targets at 2-minute cadence by the Science Processing Operations Center (SPOC) at NASA Ames (Jenkins *et al.*, 2016). The SPOC pipeline performs pixel-level calibration, aperture photometry, and systematic error correction, producing two flux columns per target: Simple Aperture Photometry (SAP) flux, which is the raw summed pixel flux within the optimal aperture, and Pre-search Data Conditioning SAP (PDCSAP) flux, which applies corrections for instrumental systematics including pointing drift, focus changes, and thermal variations (Stumpe *et al.*, 2012).

All TESS data products are stored in the FITS format and distributed through the Mikulski Archive for Space Telescopes (MAST). As of 2024, TESS has completed its primary mission (Sectors 1–26), its first extended mission (Sectors 27–55), and is continuing its second extended mission, having observed over 85% of the sky and identified more than 7,000 TESS Objects of Interest (TOIs) (Guerrero *et al.*, 2021).

2.1.2. The FITS Data Format

The Flexible Image Transport System (FITS) is the standard data format in astronomy, defined by the International Astronomical Union (IAU) FITS Working Group (Pence *et al.*, 2010). A FITS file consists of one or more Header Data Units (HDUs), each containing an ASCII header of keyword-value pairs followed by an optional data array. The primary HDU typically contains image data or is empty, while extension HDUs can hold binary tables, ASCII tables, or additional images.

TESS light curve files use binary table extensions to store time-series data. The relevant columns for this project are: TIME (Barycentric TESS Julian Date), SAP_FLUX (raw aperture photometry), PDCSAP_FLUX (corrected flux), PDCSAP_FLUX_ERR (flux uncertainty), and QUALITY (bitfield flags indicating data quality issues such as cosmic ray hits, spacecraft attitude anomalies, or stray light contamination) (Tenenbaum and Jenkins, 2018). Parsing these binary tables requires either a dedicated FITS library such as CFITSIO (Pence *et*

al., 2010) or a general-purpose binary parser that understands the FITS specification. The QUALITY column is particularly important for data pipelines: non-zero quality flags indicate data points that should be excluded or treated with caution during analysis.

The BTJD time system used by TESS is defined as Barycentric Julian Date (BJD) minus a fixed offset of 2,457,000.0 days. BJD itself corrects observed times for the light travel time difference between the observer’s position and the Solar System barycentre, ensuring that timing measurements are independent of Earth’s orbital position (Eastman, Siverd and Gaudi, 2010). Ground-based observers typically record times in UTC or Julian Date (JD), requiring conversion to BJD for direct comparison with TESS data. This conversion depends on the observer’s geographic coordinates and the target’s celestial coordinates, and introduces corrections on the order of up to 8.3 minutes (the light travel time across 1 AU).

2.1.3. The MAST Archive

The Mikulski Archive for Space Telescopes (MAST) is NASA’s primary repository for astronomical data from ultraviolet, optical, and near-infrared missions, hosted by the Space Telescope Science Institute (STScI) in Baltimore (Space Telescope Science Institute, 2024). MAST provides both a web portal and a programmatic API for querying and retrieving data. The API accepts HTTP POST requests to <https://mast.stsci.edu/api/v0/invoke> with a JSON body specifying the service name, parameters, and optional filters.

MAST’s API returns data in a columnar format: the response JSON contains separate arrays for column names, data types, and row values, rather than the row-oriented JSON objects typical of REST APIs (Space Telescope Science Institute, 2024). This design is efficient for transmitting large tabular datasets but requires client-side transformation to reconstruct records. For the Nyx platform, the relevant MAST services include:

- **Mast.Name.Lookup:** resolves a target name (e.g., “pi Mensae”) to celestial coordinates (right ascension and declination) and canonical identifiers.
- **Mast.Caom.Filtered.Position:** searches the Common Archive Observation Model (CAOM) for observations at a given sky position, filtered by mission, instrument, and data type.
- **Mast.Caom.Products:** retrieves the data products (FITS files) associated with a specific observation, including light curve files, target pixel files, and data validation reports.

The multi-step workflow — name resolution, observation search, product discovery, file download — is what the Nyx data ingestion pipeline automates.

2.2. Existing Platforms and Tools

2.2.1. Lightcurve

Lightcurve is an open-source Python package for analysing time-series data from the Kepler and TESS missions (Lightcurve Collaboration *et al.*, 2018). It provides a high-level API for downloading light curves and target pixel files from MAST, performing common

operations such as flattening (removing long-term trends), folding (phase-folding on a known period), and periodogram analysis. `Lightcurve` wraps the `astroquery.mast` module for data retrieval and `astropy` for FITS parsing.

While `Lightcurve` significantly lowers the barrier to TESS data analysis for Python users, it is a programmatic library, not a web application. It requires a local Python environment, familiarity with Jupyter notebooks or scripting, and offers no mechanism for managing user observations or comparing them against space-based data. It also does not persist any data; each session re-downloads from MAST.

2.2.2. Exoplanet Transit Database

The Exoplanet Transit Database (ETD), operated by the Czech Astronomical Society, is a web platform where amateur observers can upload transit observations of known exoplanets (Poddany, Brat and Pejcha, 2010). Observers submit their light curves along with metadata (telescope, camera, filter, observing conditions), and the ETD fits a transit model to estimate mid-transit time, depth, and duration. The platform maintains a catalogue of transit timing variations (TTVs) derived from submitted observations.

The ETD addresses the community observation side of the problem but does not integrate space-based data. Users cannot view TESS light curves alongside their own observations, and the platform does not provide data ingestion from MAST. Its focus is on transit timing rather than general photometric comparison.

2.2.3. AAVSO International Database

The American Association of Variable Star Observers (AAVSO) maintains one of the largest databases of variable star observations, with over 43 million measurements submitted by observers worldwide (Kafka, 2021). The AAVSO provides standardised reporting formats, comparison star sequences, and tools for data submission and retrieval.

The AAVSO’s scope is variable star monitoring — tracking brightness changes over time to study stellar behaviour. It is not designed for exoplanet transit detection or for cross-referencing with space-based mission data. Its data model centres on individual brightness estimates rather than continuous time-series, and it does not offer integrated visualisation of space-based reference light curves.

2.2.4. ExoFOP

The Exoplanet Follow-up Observing Program (ExoFOP) is a web-based platform operated by the NASA Exoplanet Science Institute (NExSci) that facilitates coordination of follow-up observations for TESS Objects of Interest (NASA Exoplanet Science Institute, 2019). ExoFOP aggregates community-contributed observations, imaging, and spectroscopy for candidate exoplanets identified by TESS. It provides a target-centric view with links to MAST data, published parameters, and follow-up status.

ExoFOP is the closest existing platform to Nyx in concept, but it serves a different audience and purpose. It is designed for the professional follow-up community coordinating confir-

mation observations, not for amateur observers comparing their data against TESS light curves. It does not provide light curve visualisation tools, does not accept photometric time-series uploads from amateurs, and does not offer a general-purpose observation management system.

2.3. Data Pipeline Architecture

2.3.1. ETL and ELT Patterns

Data pipelines for ingesting external data typically follow either the Extract, Transform, Load (ETL) or Extract, Load, Transform (ELT) pattern (Kimball and Ross, 2013). In ETL, data is extracted from the source, transformed into the target schema in a staging area, and then loaded into the destination database. In ELT, raw data is loaded first and transformations are performed within the database using SQL or stored procedures.

The Nyx data ingestion pipeline follows an ETL approach: MAST API responses (JSON) and FITS files (binary) are extracted from NASA’s servers, transformed into domain entities (targets, observations, light curve points) in application code, and loaded into PostgreSQL. This approach is preferable for this use case because the transformation logic — parsing columnar JSON, reading FITS binary table extensions, converting time systems, filtering on quality flags — is complex and benefits from the expressiveness of a general-purpose programming language rather than SQL.

A key consideration for astronomical data pipelines is idempotency: re-running the pipeline for a given target should not create duplicate records. This is achieved by using natural keys (MAST observation IDs) as unique constraints, allowing the pipeline to safely re-ingest data without manual deduplication (Kleppmann, 2017).

2.3.2. Batch vs. Real-Time Processing

Data ingestion can be implemented as batch processing (periodic bulk imports), real-time streaming (continuous ingestion as data becomes available), or on-demand (triggered by user action) (Kleppmann, 2017). MAST data updates are infrequent — new TESS sectors are released every 27 days — making batch or on-demand processing appropriate. Real-time streaming would add unnecessary complexity for a data source with this update cadence.

The Nyx platform uses on-demand ingestion: when a user searches for a target, the system queries MAST, retrieves observation metadata, and caches it in PostgreSQL. Subsequent requests for the same target are served from the database without re-querying MAST. FITS light curve data is fetched and parsed only when explicitly requested, as these files can be several megabytes each and the parsing is computationally expensive relative to metadata queries.

For long-running ingestion tasks such as FITS download and parsing, the backend processes the work asynchronously and reports progress to the frontend via Server-Sent Events (SSE). SSE is a W3C standard that enables a server to push updates to a client over a single long-lived HTTP connection (W3C, 2015). Unlike WebSockets, SSE is unidirectional

(server to client only), uses standard HTTP, and is automatically handled by the browser's EventSource API. This makes it well-suited for progress reporting where the client only needs to receive updates, not send messages.

2.4. Software Architecture

2.4.1. Clean Architecture

Clean Architecture, proposed by Martin (2017), organises software into concentric layers with a strict dependency rule: source code dependencies must point inward, from outer layers (frameworks, UI, databases) toward inner layers (business rules, domain entities) (Martin, 2017). The innermost layer contains enterprise business rules (domain entities), surrounded by application business rules (use cases), then interface adapters (controllers, presenters, gateways), and finally frameworks and drivers (web frameworks, databases, external APIs).

This architecture provides several benefits for the Nyx project. The domain layer — containing entities such as `Target`, `Tess0bservation`, and `LightCurvePoint` — has no dependencies on the web framework (Drogon), the database (PostgreSQL), or external APIs (MAST). Repository interfaces are defined in the domain layer and implemented in the infrastructure layer, allowing the database implementation to be changed without modifying business logic. This separation also enables unit testing of application services with mock repositories, without requiring a running database or network access.

The dependency inversion principle is central to this approach: high-level modules (services) depend on abstractions (repository interfaces), not on concrete implementations (PostgreSQL queries) (Martin, 2000). In C++, this is implemented using pure virtual classes (interfaces) with concrete implementations provided via constructor injection.

2.4.2. Hexagonal Architecture

Hexagonal Architecture, also known as Ports and Adapters, was proposed by Cockburn (2005) and is closely related to Clean Architecture (Cockburn, 2005). It distinguishes between the application core (domain logic) and the external world (databases, APIs, UIs) through defined ports (interfaces) and adapters (implementations). Primary ports are driven by external actors (e.g., an HTTP request triggering a use case), while secondary ports are driven by the application (e.g., the application querying a database).

In the Nyx backend, the MAST API client is a secondary adapter: the application service defines an `IMastClient` interface (port), and the infrastructure layer provides a concrete implementation that makes HTTP requests to MAST. Similarly, `IUserRepository` is a port with `PostgresUserRepository` as its adapter. This pattern allows the application to be tested with in-memory or mock adapters, and makes it straightforward to replace any external dependency.

2.5. Backend Technologies

2.5.1. C++ for Web Services

C++ is not a conventional choice for web application development, where interpreted languages (Python, JavaScript) and garbage-collected compiled languages (Go, Java) dominate. However, C++ offers deterministic memory management through RAII (Resource Acquisition Is Initialisation), zero-cost abstractions, and direct hardware access, making it suitable for performance-sensitive workloads (Stroustrup, 2013). Modern C++ (C++17 and later) provides features that significantly improve developer productivity: `std::optional` for nullable values, `std::variant` for type-safe unions, `std::expected` (C++23) for error handling without exceptions, structured bindings, and `constexpr` for compile-time evaluation (cppreference.com, 2024).

For the Nyx platform, C++ is chosen primarily for educational value — demonstrating that a modern, well-architected web service can be built in C++ — and for the performance characteristics relevant to parsing large FITS binary files and processing thousands of time-series data points per request.

2.5.2. Drogon Web Framework

Drogon is a C++17/20 HTTP application framework built on a non-blocking I/O event loop architecture (An, 2024). It supports HTTP/1.1 and HTTP/2, WebSocket, and provides an integrated ORM for database access. Drogon uses a multi-threaded event loop model similar to Node.js but with OS-level threads: incoming connections are distributed across event loops, each running on its own thread, with non-blocking I/O operations (database queries, HTTP client requests) handled asynchronously via callbacks or coroutines.

Drogon’s filter system provides middleware-like functionality: filters are classes that can intercept requests before they reach a controller, enabling cross-cutting concerns such as authentication, rate limiting, CSRF validation, and request logging. Controllers are classes with methods annotated via macros (`ADD_METHOD_TO`, `METHOD_ADD`) that map HTTP routes to handler functions.

The framework’s built-in ORM supports PostgreSQL, MySQL, and SQLite, providing both synchronous and asynchronous query execution. For the Nyx project, synchronous queries are used within request handlers that are already running on event loop threads, as the non-blocking I/O layer handles concurrency at the connection level.

2.6. Database Design for Time-Series Data

2.6.1. PostgreSQL for Scientific Data

PostgreSQL is a widely used open-source relational database with strong support for scientific workloads (The PostgreSQL Global Development Group, 2024). Its relevant features for the Nyx platform include: support for UUID primary keys (eliminating the need for sequential ID generation in distributed scenarios), `TIMESTAMPTZ` for timezone-aware time-

stamps, `DOUBLE PRECISION` for floating-point astronomical measurements, partial indexes for sparse columns (e.g., indexing `google_id` only where non-null), and `ON CONFLICT` clauses for upsert operations that enable idempotent data ingestion.

For time-series light curve data, the key performance consideration is range query efficiency: a typical frontend request retrieves all data points for a given observation, potentially spanning thousands of rows. A composite index on `(tess_observation_id, time)` enables efficient range scans for this access pattern. PostgreSQL’s MVCC (Multi-Version Concurrency Control) ensures that long-running read queries for light curve visualisation do not block concurrent write operations during data ingestion (The PostgreSQL Global Development Group, 2024).

2.6.2. Alternatives Considered

TimescaleDB, a PostgreSQL extension purpose-built for time-series data, provides automatic partitioning (hypertables), time-based data retention policies, and optimised aggregation functions (Timescale, Inc., 2024). While these features would benefit a system processing millions of data points across many targets, the Nyx platform’s data volume — typically 10,000 to 50,000 points per TESS light curve, with hundreds of targets — does not justify the additional operational complexity. Standard PostgreSQL with appropriate indexing is sufficient for this scale.

InfluxDB and other dedicated time-series databases were also considered but rejected because they lack relational modelling capabilities needed for the platform’s non-time-series data (users, equipment, observation sessions) and would require a polyglot persistence strategy with increased operational overhead (Kleppmann, 2017).

2.7. Authentication and Security

2.7.1. JWT-Based Authentication

JSON Web Tokens (JWT) provide a stateless authentication mechanism where the server issues a signed token containing user claims, and the client presents this token with each request (Jones, Bradley and Sakimura, 2015). The server verifies the token’s signature without querying a database, reducing latency for authenticated requests. JWTs consist of three Base64URL-encoded parts: a header (algorithm and token type), a payload (claims such as user ID, email, and expiration time), and a signature computed using HMAC-SHA256 or RSA.

The stateless nature of JWTs introduces a revocation challenge: since the server does not track issued tokens, a compromised token remains valid until expiration. The standard mitigation is a dual-token strategy: a short-lived access token (typically 15 minutes) for API authentication, and a longer-lived refresh token (typically 7 days) stored server-side for issuing new access tokens (Jones, Bradley and Sakimura, 2015). Refresh token rotation — issuing a new refresh token with each use and invalidating the previous one — provides reuse detection: if a revoked token is presented, the entire token family can be invalidated, forcing re-authentication (Lodderstedt *et al.*, 2020).

2.7.2. OAuth 2.0

OAuth 2.0 is an authorisation framework that enables third-party applications to obtain limited access to a user’s account on another service (Hardt, 2012). The Authorization Code Grant flow, used by the Nyx platform for Google sign-in, involves four steps: (1) the frontend redirects the user to Google’s consent screen, (2) Google redirects back with an authorisation code, (3) the backend exchanges the code for tokens by calling Google’s token endpoint, and (4) the backend decodes the ID token to extract user information (email, name, Google user ID).

The ID token returned by Google is a JWT containing claims about the authenticated user. Since it is received directly from Google’s servers over HTTPS (not from the client), signature verification is not strictly necessary — the transport layer guarantees authenticity (Sakimura *et al.*, 2014). The backend extracts the sub (subject, Google’s unique user ID), email, email_verified, and name claims to create or identify the user account.

2.7.3. CSRF Protection

When authentication state is carried in cookies (as with HttpOnly refresh tokens), the application becomes vulnerable to Cross-Site Request Forgery (CSRF) attacks, where a malicious website submits requests to the application using the victim’s cookies (OWASP Foundation, 2024a). The double-submit cookie pattern mitigates this: the server generates a random CSRF token, sends it as both a cookie (automatically attached by the browser) and in the response body (stored by the frontend in JavaScript). On subsequent mutating requests (POST, PUT, DELETE), the frontend sends the token in a custom HTTP header (X-CSRF-Token). The server compares the cookie value against the header value; since a cross-origin attacker cannot read cookies from another domain, they cannot construct the matching header (OWASP Foundation, 2024a).

The comparison must use constant-time comparison (e.g., `sodium_memcmp` from `libsodium`) to prevent timing side-channel attacks that could allow an attacker to determine the token value one character at a time (Bernstein, 2012).

2.7.4. Password Hashing

Secure password storage requires a computationally expensive, memory-hard hashing algorithm to resist brute-force and hardware-accelerated attacks. Argon2, the winner of the Password Hashing Competition in 2015, provides three variants: Argon2d (data-dependent memory access, resistant to GPU attacks), Argon2i (data-independent memory access, resistant to side-channel attacks), and Argon2id (a hybrid recommended for password hashing) (Biryukov, Dinu and Khovratovich, 2016). Argon2id with parameters of at least 64 MB memory, 3 iterations, and 1 degree of parallelism is recommended by OWASP for password hashing (OWASP Foundation, 2024b).

2.8. API Design

2.8.1. REST Principles

Representational State Transfer (REST) is an architectural style for distributed systems defined by Fielding (2000) in his doctoral dissertation (Fielding, 2000). REST defines six constraints: client-server separation, statelessness, cacheability, a uniform interface, layered system, and optional code-on-demand. In practice, RESTful APIs map HTTP methods to operations on resources identified by URIs: GET for retrieval, POST for creation, PUT/PATCH for updates, and DELETE for removal.

The Nyx API follows REST conventions with versioned routes (`/api/v1/...`), plural noun resource names (`/targets`, `/observations`), and a consistent JSON response envelope containing data and meta (correlation ID and timestamp) fields. Error responses follow a structured format with machine-readable error codes, human-readable messages, and optional field-level validation details.

2.8.2. Cursor-Based Pagination

For list endpoints returning potentially large result sets, the API uses cursor-based pagination rather than offset-based pagination. Offset pagination (`LIMIT 20 OFFSET 100`) suffers from performance degradation on large datasets (the database must scan and discard offset rows) and inconsistency when rows are inserted or deleted between page requests (Winand, 2012). Cursor pagination uses an opaque token encoding the last-seen row's sort key, allowing the database to seek directly to the next page via an indexed range scan. This provides consistent $O(\log n)$ performance regardless of page depth.

2.9. Frontend Technologies

2.9.1. Next.js

Next.js is a React-based framework that provides server-side rendering (SSR), static site generation (SSG), file-system-based routing, and API routes (Vercel, 2024). For the Nyx platform, Next.js is used for the frontend application, providing: server-side rendering for initial page loads (improving perceived performance and SEO), client-side navigation for subsequent interactions, and built-in support for environment variable management and middleware.

The App Router architecture introduced in Next.js 13 uses React Server Components by default, allowing components to render on the server without sending their JavaScript to the client. This reduces bundle size and improves performance for data-heavy pages such as the target catalogue browser. Client components, marked with `"use client"`, are used for interactive elements such as the light curve viewer.

2.9.2. Data Visualisation

Interactive light curve visualisation is a core feature of the Nyx frontend. The requirements include: rendering tens of thousands of data points without performance degradation, zoom

and pan interactions, tooltips showing individual point values, and overlaying multiple datasets (TESS and ground-based) on the same axes.

D3.js is the foundational library for data-driven visualisation in web browsers, providing low-level primitives for bindings between data and DOM elements (Bostock, Ogievetsky and Heer, 2011). However, for the Nyx use case, a higher-level charting library built on Canvas or WebGL rendering is more appropriate, as SVG-based rendering (used by D3 for bindings) degrades in performance beyond approximately 5,000 points. Libraries such as Chart.js (Canvas-based) and Plotly.js (built on D3 with WebGL support for scatter plots) provide the required performance characteristics while offering built-in zoom, pan, and tooltip interactions (Plotly Technologies Inc., 2024).

2.10. Summary

The literature establishes several key points that inform the design of the Nyx platform. TESS produces well-documented, publicly available time-series data in a standard format (FITS), accessible through the MAST API. Existing tools either require programming expertise (Lightkurve), focus on narrow use cases (ETD for transit timing, AAVSO for variable stars), or serve professional rather than amateur audiences (ExoFOP). No platform integrates space-based data retrieval, ground-based observation management, and comparative visualisation in a single web application.

The technical literature supports the architectural decisions made in this project: clean architecture for separation of concerns and testability, PostgreSQL for combined relational and time-series storage, JWT with refresh token rotation for stateless authentication, and on-demand ETL for data ingestion from MAST. The following chapter translates these findings into concrete requirements for the Nyx system.

3. Requirements Analysis

4. System Design

5. Implementation

6. Testing and Validation

7. Conclusion

Bibliography

- An, T. (2024) “Drogon: A C++14/17/20 based HTTP web application framework.”
- Bernstein, D.J. (2012) “The NaCl Networking and Cryptography library,” *Document ID: 5bbef7e* [Preprint].
- Biryukov, A., Dinu, D. and Khovratovich, D. (2016) “Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications,” *IEEE European Symposium on Security and Privacy*, pp. 292–302. Available at: <https://doi.org/10.1109/EuroSP.2016.31>.
- Borucki, W.J. *et al.* (2010) “Kepler Planet-Detection Mission: Introduction and First Results,” *Science*, 327(5968), pp. 977–980. Available at: <https://doi.org/10.1126/science.1185402>.
- Bostock, M., Ogievetsky, V. and Heer, J. (2011) “D3: Data-Driven Documents,” *IEEE Transactions on Visualization and Computer Graphics*, 17(12), pp. 2301–2309. Available at: <https://doi.org/10.1109/TVCG.2011.185>.
- Cockburn, A. (2005) “Hexagonal Architecture.”
- Conti, D.M. and Conti, D. (2018) *A Practical Guide to Exoplanet Observing*. AAS.
- cppreference.com (2024) “C++ reference.”
- Eastman, J., Siverd, R. and Gaudi, B.S. (2010) “Achieving Better Than 1 Minute Accuracy in the Heliocentric and Barycentric Julian Dates,” *Publications of the Astronomical Society of the Pacific*, 122(894), pp. 935–946. Available at: <https://doi.org/10.1086/655938>.
- Fielding, R.T. (2000) *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation.
- Fitzgerald, M.T. *et al.* (2014) “A Review of High School Level Astronomy Student Research Projects Over the Last Two Decades,” *Publications of the Astronomical Society of Australia*, 31, p. e37. Available at: <https://doi.org/10.1017/pasa.2014.30>.
- Gary, B.L. (2007) *Exoplanet Observing for Amateurs*. Reductionist Publications.
- Guerrero, N.M. *et al.* (2021) “The TESS Objects of Interest Catalog from the TESS Prime Mission,” *The Astrophysical Journal Supplement Series*, 254(2), p. 39. Available at: <https://doi.org/10.3847/1538-4365/abefe1>.
- Hardt, D. (2012) “The OAuth 2.0 Authorization Framework”. Available at: <https://doi.org/10.17487/RFC6749>.
- Hessman, F.V. (2004) “The Longitude Problem for Ground-Based Observations of Transiting Planets,” *Astrophysics and Space Science*, 292(1–4), pp. 471–479. Available at: <https://doi.org/10.1023/B:ASTR.0000045049.94815.4f>.

- Jenkins, J.M. *et al.* (2016) “The TESS science processing operations center,” in *Software and Cyberinfrastructure for Astronomy IV*. SPIE, p. 99133E. Available at: <https://doi.org/10.1117/12.2233418>.
- Jones, M., Bradley, J. and Sakimura, N. (2015) “JSON Web Token (JWT)”. Available at: <https://doi.org/10.17487/RFC7519>.
- Kafka, S. (2021) “Observations from the AAVSO International Database.”
- Kimball, R. and Ross, M. (2013) *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3rd ed. John Wiley & Sons.
- Kleppmann, M. (2017) *Designing Data-Intensive Applications*. O'Reilly Media.
- Lightkurve Collaboration *et al.* (2018) “Lightkurve: Kepler and TESS time series analysis in Python.”
- Lodderstedt, T. *et al.* (2020) “OAuth 2.0 Security Best Current Practice.”
- Martin, R.C. (2000) *Design Principles and Design Patterns*. Object Mentor.
- Martin, R.C. (2017) *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- NASA Exoplanet Science Institute (2019) “Exoplanet Follow-up Observing Program (ExoFOP).”
- OWASP Foundation (2024a) “Cross-Site Request Forgery Prevention Cheat Sheet.”
- OWASP Foundation (2024b) “Password Storage Cheat Sheet.”
- Pence, W.D. *et al.* (2010) “Definition of the Flexible Image Transport System (FITS), version 3.0,” *Astronomy & Astrophysics*, 524, p. A42. Available at: <https://doi.org/10.1051/0004-6361/201015362>.
- Plotly Technologies Inc. (2024) “Plotly.js Open Source Graphing Library for JavaScript.”
- Poddany, S., Brat, L. and Pejcha, O. (2010) “Exoplanet Transit Database. Reduction and processing of the photometric data of exoplanet transits,” *New Astronomy*, 15(3), pp. 297–301. Available at: <https://doi.org/10.1016/j.newast.2009.09.001>.
- The PostgreSQL Global Development Group (2024) “PostgreSQL 16 Documentation.”
- Ricker, G.R. *et al.* (2015) “Transiting Exoplanet Survey Satellite (TESS),” *Journal of Astronomical Telescopes, Instruments, and Systems*, 1(1), p. 14003. Available at: <https://doi.org/10.1117/1.JATIS.1.1.014003>.
- Sakimura, N. *et al.* (2014) “OpenID Connect Core 1.0.”
- Space Telescope Science Institute (2024) “Mikulski Archive for Space Telescopes (MAST).”

- Stroustrup, B. (2013) *The C++ Programming Language*. 4th ed. Addison-Wesley.
- Stumpe, M.C. *et al.* (2012) “Kepler Presearch Data Conditioning I — Architecture and Algorithms for Error Correction in Kepler Light Curves,” *Publications of the Astronomical Society of the Pacific*, 124(919), pp. 985–999. Available at: <https://doi.org/10.1086/667698>.
- Tenenbaum, P. and Jenkins, J.M. (2018) “TESS Science Processing Operations Center (SPOC),” in *Software and Cyberinfrastructure for Astronomy V*. SPIE, p. 107070K. Available at: <https://doi.org/10.1117/12.2312590>.
- Timescale, Inc. (2024) “TimescaleDB Documentation.”
- Vercel (2024) “Next.js Documentation.”
- W3C (2015) “Server-Sent Events.”
- Winand, M. (2012) *SQL Performance Explained*. Markus Winand.
- Winn, J.N. (2010) “Transits and Occultations,” *arXiv preprint arXiv:1001.2010* [Preprint]. Available at: <https://doi.org/10.48550/arXiv.1001.2010>.